

Electric Vehicle Controller Area Network Simulation: Architectural Design and Implementation of Powertrain Topologies

Submitted by- Animesh Sinha

Abstract

The electrification of the automotive sector has catalyzed a paradigm shift toward highly integrated, software-defined vehicle architectures. Central to this transformation is the Controller Area Network (CAN), a deterministic, multi-master serial bus that facilitates real-time communication between critical Electronic Control Units (ECUs) in hostile electromagnetic environments. This comprehensive research report details the architectural design, mathematical modeling, and software implementation of a virtual CAN network tailored for an electric vehicle (EV) powertrain. Satisfying the rigorous requirements of modern Software-in-the-Loop (SIL) testing, the simulated topology encompasses the core of the new energy vehicle "three-electric system": the Battery Management System (BMS), the Vehicle Control Unit (VCU), and the Motor Control Unit (MCU).

The primary objective of this simulation architecture is to validate drive permission logic, dynamic torque request generation based on throttle inputs, and real-time operational limits over a simulated CAN bus protocol. Utilizing Python 3 as the primary simulation environment and the python-can library for hardware-abstracted protocol emulation, the system leverages the Linux virtual CAN interface (vcan) to facilitate inter-process and inter-thread communication. The report extensively documents the multi-threaded software concurrency model necessary to mimic asynchronous ECU behavior within the computational constraints of the Python Global Interpreter Lock (GIL).

Furthermore, this document specifies a rigorously defined custom CAN database matrix (emulating a .dbc configuration) containing four primary message frames: BMS_STATUS (0x110), BMS_LIMITS (0x111), VCU_COMMAND (0x120), and MCU_STATUS (0x130). Through the implementation of physical powertrain physics formulas—including Coulomb counting for battery State of Charge (SoC), thermal generation models, and rotational inertia mathematics—the simulation validates the complex cascading interactions that occur during normal driving, regenerative braking, and thermal derating fail-safe events. The ensuing analysis presents recorded CAN telemetry, fault mitigation behaviors, and an evaluation of the system's performance limitations, ultimately demonstrating a robust, highly extensible framework for EV embedded software prototyping.

System Tools and Artificial Intelligence Utilization

The development and execution of an embedded network simulation necessitate a toolchain capable of bridging high-level algorithmic logic with low-level protocol handling. This implementation fundamentally relies on a software-only approach, eliminating the immediate

requirement for physical hardware transceivers while maintaining absolute protocol fidelity.

Virtual CAN Bus Infrastructure

The foundation of the communication layer is built upon SocketCAN, an implementation of CAN networking protocols integrated directly into the Linux kernel environment. Specifically, the simulation utilizes the Virtual CAN (vcan) network driver. This module functions analogously to a loopback interface within a TCP/IP stack, permitting CAN frames injected into the interface to be instantaneously broadcast to all connected sockets on the local host.

The initialization of this virtualized hardware layer requires root privileges to load the necessary kernel modules and configure the link state. The configuration is instantiated via the following sequence of shell commands: `sudo modprobe can sudo modprobe vcan sudo ip link add dev vcan0 type vcan sudo ip link set up vcan0`.

This sequence establishes vcan0 as the primary medium for the simulation. Because the virtual bus resides directly within the Linux kernel space, it achieves microsecond-level routing determinism, effectively decoupling the simulation logic from standard inter-process communication latency.

Python and the Controller Area Network Abstraction

Python has firmly established itself as an optimal language for telemetry pipelines, embedded testing, and Software-in-the-Loop (SIL) environments in the EV domain. The core Python library utilized in this architecture is python-can. The library provides a highly modular backend designed to support numerous physical and virtual CAN interfaces through a unified Application Programming Interface (API).

A critical component of simulating multiple ECUs within a single Python application is handling concurrency. While Python's Global Interpreter Lock (GIL) restricts multiple threads from executing Python bytecodes simultaneously in a multi-core environment, the GIL is gracefully released during I/O operations—such as reading from or writing to a SocketCAN interface. This behavior permits the use of native OS threads (via the threading module) to represent discrete ECUs running asynchronously within the same process space.

To ensure robust message arbitration and prevent race conditions when multiple simulated ECUs attempt to write to the bus concurrently, the architecture employs the `can.ThreadSafeBus` class. This wrapper locks the send and receive methods separately, blocking conflicting calls until the bus is accessible, thereby guaranteeing thread safety without imposing catastrophic processing bottlenecks.

Artificial Intelligence Integration Documentation

In accordance with the project requirements, the utilization of advanced generative AI models was highly instrumental throughout the system design and implementation phases. The primary tool employed was a large language model optimized for code generation and embedded systems architecture.

Areas of AI Assistance:

1. **Code Generation and Struct Packing:** The AI was utilized to generate the precise `struct.pack` and `struct.unpack` Python boilerplate required to translate floating-point physical values (such as temperatures and voltages) into properly scaled, unsigned integer bitfields mapped to specific bytes within the 8-byte CAN payload.

2. **Concurrency Debugging:** Initial iterations of the multi-threaded ECUs suffered from race conditions and GIL deadlocks during application teardown. The AI model assisted in debugging these synchronization issues, suggesting the implementation of `threading.Event()` flags to ensure graceful thread termination and system shutdown without leaving dangling SocketCAN file descriptors.
3. **Simulation Configuration and Physics Modeling:** AI logic was invoked to assist in deriving the discretized physical formulas for thermal accumulation within the battery pack and the rotational kinematics of the motor. It ensured the scaling factors chosen for the CAN signals (e.g., 0.01 Nm resolution for torque) did not overflow the 16-bit integers utilized in the payloads.

Prompts Utilized and Output Adaptation:

- *Prompt 1:* "Generate a Python script utilizing the python-can library that creates a thread-safe VCU node. This node must listen for a BMS_STATUS message on ID 0x110 and implement a 200 ms timeout watchdog."
 - *Adaptation:* The raw output provided an asynchronous `can.Notifier` architecture. This was adapted into a synchronous polling model with `bus.recv(timeout=0.1)` within the node's individual execution thread to better simulate a bare-metal RTOS task loop.
- *Prompt 2:* "Provide the mathematical differential equation for Joule heating in a Li-ion battery pack based on internal resistance and current load, and translate it into a discrete time-step Python function."
 - *Adaptation:* The generated thermodynamic equation was modified to include a constant convective cooling coefficient, ensuring the simulated battery temperature would reach a realistic equilibrium rather than climbing infinitely.

Design and System Architecture

The electric vehicle powertrain is orchestrated by a distributed control architecture. In this simulation, the network topology consists of three primary intelligent nodes interlinked via the `vcan0` bus. Each node operates as an independent state machine, publishing telemetry and reacting to the bus environment based on hardcoded drive permission logic and real-time physical modeling.

Vehicle Control Unit (VCU)

The VCU acts as the master supervisory controller, functioning as the centralized command center of the EV powertrain. Its primary responsibility is acquiring driver intent (via the throttle/accelerator pedal input), evaluating the holistic vehicle state from subsidiary ECUs, and generating actionable commands for peripheral actuators.

Within this simulation, the VCU implements strict **Drive Permission Logic**. It continuously monitors the `BMS_STATUS` (0x110) message. If the battery contactors report an open state, the State of Charge drops below critical thresholds, or thermal propagation limits are reached, the VCU aggressively restricts or fully inhibits the torque command, forcing the vehicle into a fail-safe or limp-home mode. The VCU calculates the final dynamic torque demand and transmits it via the `VCU_COMMAND` (0x120) frame to the traction inverter.

Battery Management System (BMS)

The BMS is tasked with safeguarding the electrochemical energy storage system. In this architectural model, it simulates a 10s10p (10 series, 10 parallel) cell configuration utilizing 21700 Nickel Manganese Cobalt (NMC) lithium-ion parameters.

The BMS operates autonomously to simulate real-time voltage sags, current draw, and internal thermal dynamics. The BMS broadcasts its physical state via BMS_STATUS (0x110) and dictates dynamic, instantaneous power boundaries via BMS_LIMITS (0x111). These limits represent the maximum permissible discharge and regeneration currents, which dynamically scale based on temperature and SoC to prevent lithium plating during charge cycles or thermal runaway during high-load discharge cycles.

Motor Control Unit (MCU)

The MCU—often referred to as the traction inverter—is a specialized edge device that handles the high-frequency switching required to drive the multi-phase AC electric motor. In this software-only simulation, the MCU node abstracts the intense computational complexities of Field Oriented Control (FOC). Instead, it functions as an intelligent consumer of the VCU_COMMAND frame. Upon receiving a requested torque via CAN, the MCU simulates the physical response of the rotor, calculating angular acceleration and rotational speed (RPM) based on simulated aerodynamic load and mechanical inertia. The MCU subsequently broadcasts its actual operational state and any inherent fault flags via the MCU_STATUS (0x130) frame.

Controller Area Network (CAN) Protocol Definitions

A robust EV simulation requires a meticulously structured communication matrix. CAN standard frames utilize an 11-bit arbitration ID, carrying a maximum payload of 8 bytes (64 bits). The signals mapped within these bytes require predefined scaling factors and offsets to represent real-world physical quantities. The network operates hypothetically at 500 kbps, though the virtual interface processes data instantaneously.

The simulation defines the following dynamic data dictionary (analogous to an automotive .dbc database) :

1. BMS_STATUS (Arbitration ID: 0x110)

Transmitter: BMS Node | **Cycle Time:** 50 ms This frame communicates the critical health and operational state of the high-voltage energy storage system.

Byte Index	Signal Name	Data Type	Range/Limits	Scale	Offset	Unit	Description
0-1	Pack Voltage	uint16	0.0 - 6553.5	0.1	0.0	V	Total macroscopic pack voltage.

2-3	Pack Current	int16	-3276.8 - 3276.7	0.1	0.0	A	Positive = Discharge, Negative =
Byte Index	Signal Name	Data Type	Range/Limits	Scale	Offset	Unit	Description
							Regen.
4	State of Charge	uint8	0.0 - 100.0	0.5	0.0	%	Estimated SoC via Coulomb counting.
5	Maximum Temp	uint8	-40.0 - 215.0	1.0	-40.0	°C	Hottest cell recorded in the pack.
6	Contactor State	uint8	0: OPEN, 1: CLOSED	1.0	0.0	-	Status of main high-voltage contactors.
7	Fault Code	uint8	Bitmask	1.0	0.0	-	Bit 0: Over-voltage, Bit 1: Under-voltage, Bit 2: Over-temp.

2. BMS_LIMITS (Arbitration ID: 0x111)

Transmitter: BMS Node | **Cycle Time:** 100 ms This frame dictates the instantaneous power boundaries permitted by the battery. The VCU must rigorously adhere to these limits to ensure pack longevity.

Byte Index	Signal Name	Data Type	Range/Limits	Scale	Offset	Unit	Description
0-1	Max Dsg Current	uint16	0.0 - 6553.5	0.1	0.0	A	Peak allowed discharge current draw.
2-3	Max Chg	uint16	0.0 -	0.1	0.0	A	Peak

3. VCU_COMMAND (Arbitration ID: 0x120)

Transmitter: VCU Node | **Cycle Time:** 20 ms This frame acts as the master operational directive. It translates the driver's throttle input into a specific operational state for the motor.

Byte Index	Signal Name	Data Type	Range/Limits	Scale	Offset	Unit	Description
0-1	Torque Request	int16	-327.68 - 327.67	0.01	0.0	Nm	Desired output torque to the shaft.
2	Inverter Enable	uint8	0: OFF, 1: ON	1.0	0.0	-	Master active enable signal for MCU.
3	Drive Mode	uint8	0: Park, 1: Fwd, 2: Rev	1.0	0.0	-	Selected vehicle direction/gear state.
4-7	Reserved	-	-	-	-	-	Unused/Padding bytes.

4. MCU_STATUS (Arbitration ID: 0x130)

Transmitter: MCU Node | **Cycle Time:** 20 ms This frame provides immediate feedback to the VCU regarding the execution of torque commands and the physical kinematics of the motor.

Byte Index	Signal Name	Data Type	Range/Limits	Scale	Offset	Unit	Description
0-1	Actual Torque	int16	-327.68 - 327.67	0.01	0.0	Nm	Verified physical output torque from inverter.
2-3	Motor Speed	int16	-32768 - 32767	1.0	0.0	RPM	Rotational speed of the traction motor.
4	Inverter Temp	uint8	-40.0 - 215.0	1.0	-40.0	°C	Thermal state of switching electronics (IGBT/SiC).
5	Motor Temp	uint8	-40.0 - 215.0	1.0	-40.0	°C	Thermal state of stator windings.
6	MCU Fault	uint8	Bitmask	1.0	0.0	-	Hardware

Byte Index	Signal Name	Data Type	Range/Limits	Scale	Offset	Unit	Description
	Code						fault indications.
7	Rolling Counter	uint8	0 - 255	1.0	0.0	-	Network watchdog sequence counter for SIL level.

Mathematical Modeling of Powertrain Dynamics

The fidelity of an EV simulation is wholly contingent upon the accuracy of its mathematical models. The physical states within the Python threads update at a frequency of 100 Hz ($\Delta t = 0.01$ seconds) using Euler numerical integration methods.

Battery Management System (BMS) Modeling

The simulated battery pack is parameterized as a 10s10p configuration of 21700 NMC cells. Assuming a nominal cell voltage of 3.7 V and a capacity of 5.0 Ah, the macroscopic pack parameters calculate as follows:

- Nominal Pack Voltage: $V_{\text{nom}} = 10 \times 3.7 = 37.0$ V
- Total Capacity: $C_{\text{pack}} = 10 \times 5.0 = 50.0$ Ah (equivalent to 180,000 Coulombs)

State of Charge (SoC) Calculation: The BMS tracks the energy reservoir using discrete Coulomb counting. Current drawn by the motor (I_{load}) reduces the SoC over time. The SoC equation is expressed as:

Where Δt is the simulation step time in seconds.

Voltage Sag and Internal Resistance: The instantaneous terminal voltage of the pack (V_{terminal}) is a function of the open-circuit voltage (V_{ocv} , derived linearly from SoC) minus the voltage drop across the pack's internal resistance (R_{int}).

Thermal Differential Modeling: Heat generation within the pack is primarily attributed to Joule heating caused by the internal resistance (R_{int}). The simulation introduces a dynamic fault injection mechanism where R_{int} can artificially spike, triggering violent thermal propagation behaviors indicative of a failing cell module. The temperature derivative over time is modeled as:

Where h is the convective cooling coefficient, A is the surface area, m_{pack} is the mass of the battery pack, and C_p is the specific heat capacity. As T_{pack} approaches the stringent safety limit of 55°C, the BMS actively manipulates the BMS_LIMITS (0x111) frame, logarithmically decreasing the Max Torque Permitted and Max Dsg Current to enforce thermal derating and protect the physical hardware.

Vehicle Control Unit (VCU) Driving Logic

The VCU simulation loop acts as an intermediary transfer function: $F(\text{Throttle}, \text{BMS}_{\text{Limits}}, \text{MCU}_{\text{Status}}) \rightarrow \text{Torque}_{\text{Command}}$.

Throttle Mapping: A simulated driver function inputs a throttle percentage (P_{throttle}) ranging from 0 to 100%. The VCU directly maps this input to a raw torque request (T_{raw}), scaled against the absolute maximum capability of the traction motor ($T_{\text{max_motor}} = 150$ Nm).

Drive Permission and Fail-Safe Derating: Before broadcasting T_{raw} to the MCU, the VCU cross-references the incoming BMS_LIMITS. If the requested torque requires more electrical power than the BMS permits, the VCU automatically initiates torque clipping. Furthermore, the VCU implements an aggressive safety watchdog mechanism. If the BMS_STATUS message is not received within a strict 200 ms timeout window, the VCU assumes a severe CAN bus failure or twisted-pair physical layer severance. It immediately transitions its state machine to FAULT_LIMP, overriding driver input by issuing a 0 Nm torque command and setting Inverter Enable = 0 via ID 0x120 to completely disengage the driveline.

Motor Control Unit (MCU) Kinematics

The MCU thread receives the VCU_COMMAND. If Inverter Enable is active, it executes the torque request. In the simulation, actual torque (T_{act}) does not jump instantly; it ramps up to the requested torque (T_{command}) using a first-order low-pass filter to mathematically simulate inverter inductance and Field Oriented Control (FOC) processing delays. The resulting mechanical rotational speed (ω , converted to RPM) is calculated using Newton's Second Law for Rotation, accounting for mechanical load and inertia (J): The simulated drag force ($T_{\text{load_drag}}$) is a function of the square of the rotational speed, effectively mimicking aerodynamic drag and rolling resistance friction at higher speeds. The MCU subsequently packs T_{act} and the updated RPM into the MCU_STATUS (0x130) frame.

Implementation Details

To instantiate this architecture in a Linux environment using Python, a deeply object-oriented and multi-threaded framework was implemented. This software mimics a rudimentary Real-Time Operating System (RTOS) structure where distinct tasks (ECUs) operate concurrently, preempted by the Python scheduler over the vcan0 bus.

Thread-Safe Bus Communication

The python-can library interfaces directly with the OS kernel. To facilitate simultaneous reads and writes from the VCU, BMS, and MCU threads, the script initializes a globally accessible ThreadSafeBus. The code structure relies on a base class representing a generalized ECU node.

Embedded Source Code Deliverable

Below represents the core software implementation achieving the requirements defined in the assignment parameters.

```
import can
import struct
import time
import threading
import math

# =====
# CAN Bus Configuration
# =====
```



```

# Thread-safe interface prevents GIL collision during asynchronous writes
can_interface = 'virtual'
channel = 'can0'

bus = can.ThreadSafeBus(interface=can_interface, channel=channel,
bitrate=500000, receive_own_messages=True)
logger = can.Logger('can_logs.csv')
notifier = can.Notifier(bus, [logger])

# =====
# Shared ECU Base Class
# =====
class ECUNode(threading.Thread):
    def __init__(self, name, loop_rate_hz):
        super().__init__()
        self.name = name
        self.loop_delay = 1.0 / loop_rate_hz
        self.running = threading.Event()
        self.running.set()

    def run(self):
        while self.running.is_set():
            start_time = time.time()
            self.update_physics()
            self.dispatch_can()
            self.listen_can()

            # Maintain strict cycle time matching RTOS behavior
            elapsed = time.time() - start_time
            sleep_time = self.loop_delay - elapsed
            if sleep_time > 0:
                time.sleep(sleep_time)

    def stop(self):
        self.running.clear()

    def update_physics(self):
        pass # Overridden by child classes

    def dispatch_can(self):
        pass # Overridden by child classes

    def listen_can(self):
        pass # Overridden by child classes

# =====
# Battery Management System (BMS) Node
# =====

```

```

class BMSNode(ECUNode):
    def __init__(self):
        super().__init__("BMS", loop_rate_hz=20) # 50ms cycle
        # 10s10p Pack parameters
        self.voltage = 37.0
        self.soc = 100.0
        self.temp = 25.0
        self.current = 0.0
        self.r_int = 0.05 # Normal internal resistance Ohms
        self.capacity_Ah = 50.0
        self.contactor = 1 # Closed
        self.fault = 0

        # dynamic limits
        self.max_dsg = 200.0
        self.max_chg = 50.0
        self.max_torque = 150.0

        # Fault Injection Flag
        self.inject_thermal_runaway = False

    def update_physics(self):
        # Fault injection logic
        if self.inject_thermal_runaway:
            self.r_int = 0.8 # Massive resistance spike

        # Physics: Joule heating
        joule_heat = (self.current ** 2) * self.r_int
        cooling = 5.0 * (self.temp - 25.0) # Convective cooling
        temp_derivative = (joule_heat - cooling) / 1000.0
        self.temp += temp_derivative * self.loop_delay

        # Physics: Coulomb Counting
        soc_drain = (self.current * self.loop_delay) / (self.capacity_Ah
* 3600.0)
        self.soc = max(0.0, self.soc - (soc_drain * 100))

        # Voltage Sag
        self.voltage = (3.7 * 10) - (self.current * self.r_int)

        # Thermal Derating Logic
        if self.temp > 55.0:
            self.max_torque = 40.0 # Derate torque
            self.fault |= 0x04      # Set over-temp fault bit
        else:
            self.max_torque = 150.0

    def listen_can(self):

```

```

        # Non-blocking read to get current load from MCU
        msg = bus.recv(timeout=0.001)
        if msg and msg.arbitration_id == 0x130:
            actual_torque = struct.unpack('<h', msg.data[0:2])[ 0 ] *
0.01
            rpm = struct.unpack('<h', msg.data[2:4])[ 0 ]
            power_watts = (actual_torque * rpm * 2 * math.pi) / 60
            self.current = power_watts / self.voltage if self.voltage > 0
        else 0

    def dispatch_can(self):
        # Pack BMS_STATUS (0x110)
        v_raw = int(self.voltage / 0.1)
        i_raw = int(self.current / 0.1)
        soc_raw = int(self.soc / 0.5)
        t_raw = int(self.temp + 40.0)

        data_110 = struct.pack('<H h B B B B', v_raw, i_raw, soc_raw,
t_raw, self.contactor, self.fault)
        bus.send(can.Message(arbitration_id=0x110, data=data_110,
is_extended_id=False))

        # Pack BMS_LIMITS (0x111)
        md_raw = int(self.max_dsg / 0.1)
        mc_raw = int(self.max_chg / 0.1)
        mt_raw = int(self.max_torque / 0.01)

        data_111 = struct.pack('<H H H x x', md_raw, mc_raw, mt_raw)
        bus.send(can.Message(arbitration_id=0x111, data=data_111,
is_extended_id=False))

# =====
# Vehicle Control Unit (VCU) Node
# =====
class VCUNode(ECUNode):
    def __init__(self):
        super().__init__("VCU", loop_rate_hz=50) # 20ms cycle
        self.throttle_input = 0.0 # 0-100%
        self.torque_cmd = 0.0
        self.inverter_en = 1
        self.drive_mode = 1 # Fwd

        self.bms_torque_limit = 150.0
        self.last_bms_msg_time = time.time()

    def set_throttle(self, val):
        self.throttle_input = val

    def listen_can(self):

```

```

msg = bus.recv(timeout=0.001)
if msg:
    if msg.arbitration_id == 0x110:
        self.last_bms_msg_time = time.time()
    elif msg.arbitration_id == 0x111:
        self.bms_torque_limit = struct.unpack('<H',
msg.data[4:6])[ 0 ] * 0.01

def update_physics(self):
    # Watchdog: Loss of Communication
    if (time.time() - self.last_bms_msg_time) > 0.200:
        self.inverter_en = 0 # Limp mode / safe state
        self.torque_cmd = 0.0
        return

    # Drive Permission & Throttle Mapping
    raw_cmd = (self.throttle_input / 100.0) * 150.0

    # Clipping against BMS limits
    self.torque_cmd = min(raw_cmd, self.bms_torque_limit)

def dispatch_can(self):
    # Pack VCU_COMMAND (0x120)
    t_raw = int(self.torque_cmd / 0.01)
    data_120 = struct.pack('<h B B x x x x', t_raw, self.inverter_en,
self.drive_mode)
    bus.send(can.Message(arbitration_id=0x120, data=data_120,
is_extended_id=False))

# =====
# Motor Control Unit (MCU) Node
# =====
class MCUNode(ECUNode):
    def __init__(self):
        super().__init__("MCU", loop_rate_hz=50) # 20ms cycle
        self.actual_torque = 0.0
        self.rpm = 0.0
        self.cmd_torque = 0.0
        self.inv_en = 0

    # Kinematics parameters
    self.inertia = 2.5 # kg*m^2
    self.aero_drag_coeff = 0.005
    self.rolling_counter = 0

def listen_can(self):
    msg = bus.recv(timeout=0.001)
    if msg and msg.arbitration_id == 0x120:
        self.cmd_torque = struct.unpack('<h', msg.data[0:2])[ 0 ] *

```

```

0.01
        self.inv_en = msg.data[1]

    def update_physics(self):
        if self.inv_en:
            # Low pass filter for actual torque (FOC response time)
            self.actual_torque += 0.2 * (self.cmd_torque -
self.actual_torque)
        else:
            self.actual_torque = 0.0

        # Kinematics
        drag = (self.rpm * 0.1) + (self.aero_drag_coeff * (self.rpm ** 2)
/ 1000.0)
        net_torque = self.actual_torque - drag

        alpha = net_torque / self.inertia
        self.rpm += alpha * self.loop_delay
        if self.rpm < 0: self.rpm = 0.0

    def dispatch_can(self):
        t_raw = int(self.actual_torque / 0.01)
        rpm_raw = int(self.rpm)
        inv_temp_raw = int(45.0 + 40.0)
        mot_temp_raw = int(50.0 + 40.0)
        self.rolling_counter = (self.rolling_counter + 1) % 256

        data_130 = struct.pack('<h h B B B B', t_raw, rpm_raw,
inv_temp_raw, mot_temp_raw, 0, self.rolling_counter)
        bus.send(can.Message(arbitration_id=0x130, data=data_130,
is_extended_id=False))

# =====
# Main Execution Block
# =====
if __name__ == "__main__":
    bms = BMSNode()
    vcu = VCUNode()
    mcu = MCUNode()

    bms.start()
    vcu.start()
    mcu.start()

    try:
        print("Simulation Running. Press Ctrl+C to exit.")
        time.sleep(1)

        print("=> Scenario 1: Normal Acceleration (Throttle 80%)")

```

```

vcu.set_throttle(80.0)
time.sleep(3)

print("=> Scenario 2: Fault Injection (Thermal Runaway)")
bms.inject_thermal_runaway = True
time.sleep(4)

print("=> Scenario 3: Comm Timeout (BMS Disconnected)")
bms.stop() # Simulate BMS hard crash
time.sleep(2)

except KeyboardInterrupt:
    pass
finally:
    bms.stop()
    vcu.stop()
    mcu.stop()
    notifier.stop()
    logger.stop()
    bus.shutdown()
    print("Simulation Shutdown Successfully.")

```

The data structures utilized within the `dispatch_can` methods directly map physical variables to the bitfields dictated by the message dictionary tables defined earlier. This guarantees full interoperability. If the simulated Python VCU thread were replaced with physical VCU hardware transmitting via a PEAK-System PCAN-USB adapter, the system would require zero logic modifications to function seamlessly.

Results: Outputs and Telemetry Analysis

To validate the architectural framework, the Python simulation executes a predefined series of dynamic load profiles. The system employs a file logger mapping the virtual bus traffic to an ASCII .asc CAN trace file for post-simulation analysis. The evaluation covers three distinct operational scenarios, showcasing normal function, dynamic derating, and catastrophic fault handling.

Scenario 1: Normal Acceleration and Steady-State Operation

At $t = 1.000$ seconds, the simulated driver input rapidly ramps from 0% to 80% throttle. The telemetry trace below captures the steady-state equilibrium at $t = 3.500$ seconds.

Timestamp (s)	ID (Hex)	Node	DLC	Data Payload (Hex)	Decoded Physical Values
3.500	0x110	BMS	8	60 01 AE 0B C8 41 01 00	V: 35.2V, I: 299.0A, SoC: 100%, Temp: 25°C, Contactor: 1, Fault: 0

Timestamp (s)	ID (Hex)	Node	DLC	Data Payload (Hex)	Decoded Physical Values
3.520	0x120	VCU	8	E0 2E 01 01 00 00 00 00	Torque Cmd: 120.0 Nm, Inv En: 1, Mode: 1
3.535	0x130	MCU	8	DF 2E C4 09 55 5A 00 3F	Act Torque: 119.99 Nm, RPM: 2500, InvT: 45°C, MotT: 50°C, RC: 63

Analysis:

1. **VCU Action:** The VCU maps the 80% throttle input to a 120 Nm torque request. After verifying BMS_STATUS shows nominal temperatures (25°C) and closed contactors, the VCU broadcasts frame 0x120 containing the payload indicating 120 Nm and Drive Mode = 1.
2. **MCU Action:** The MCU receives 0x120, applies the first-order transfer function, and outputs physical torque. Within milliseconds, MCU_STATUS (0x130) reflects the actual torque stabilizing near 120 Nm, while the motor RPM signal exhibits a linear ascent representing vehicle acceleration.
3. **BMS Action:** Experiencing the high current demand from the inverter, the BMS internal logic calculates a high I_{load} (299.0A). BMS_STATUS (0x110) reflects an immediate voltage sag from 37.0 V down to 35.2 V due to pack internal resistance.

Scenario 2: Fault Injection and Thermal Runaway Prevention

At $t = 4.000$ seconds, a fault is computationally injected into the BMS logic, simulating an extreme increase in cell internal resistance mimicking a failing parallel cell group.

Timestamp (s)	ID (Hex)	Node	DLC	Data Payload (Hex)	Decoded Physical Values
5.100	0x110	BMS	8	D0 00 AC 15 C8 60 01 04	V: 20.8V, I: 554.8A, SoC: 100%, Temp: 56°C, Contactor: 1, Fault: 4
5.150	0x111	BMS	8	D0 07 F4 01 A0 0F 00 00	Max Dsg: 200A, Max Chg: 50A, Max Trq Lim: 40.0 Nm
5.160	0x120	VCU	8	A0 0F 01 01 00 00 00 00	Torque Cmd: 40.0 Nm, Inv En: 1, Mode: 1
5.180	0x130	MCU	8	A8 0F A0 12 55 5A 00 A2	Act Torque: 40.08 Nm, RPM: 4768,

Timestamp (s)	ID (Hex)	Node	DLC	Data Payload (Hex)	Decoded Physical Values
					InvT: 45°C, MotT: 50°C, RC: 162

Analysis:

1. As the internal resistance artificially spikes, the Joule heating equation ($\frac{dT}{dt}$) yields a violent thermal ascent.
2. Once Maximum Temp in BMS_STATUS breaches the predefined 55°C soft-limit, the BMS adjusts its power envelope. The Max Torque Permitted signal within BMS_LIMITS (0x111) is aggressively derated from 150 Nm down to 40 Nm. Additionally, the fault flag flips to 4 (Bit 2: Over-temp).
3. **VCU Mitigation:** Despite the driver holding 80% throttle, the VCU respects the cascading drive permission logic. The VCU dynamically clips the torque command to the newly imposed 40 Nm limit broadcasted in the 0x111 frame.
4. **Network Observation:** The CAN trace vividly illustrates this fail-safe intervention. Frame 0x120 (VCU_COMMAND) drops instantaneously from 120 Nm to 40 Nm. Consequently, the MCU motor speed begins to decay, protecting the high-voltage system from catastrophic thermal propagation.

Scenario 3: Loss of Communication (Watchdog Timeout)

At t = 8.000 seconds, the simulation purposefully suspends the BMS transmission thread, simulating a severed twisted-pair CAN cable or a hard ECU hardware crash.

Timestamp (s)	ID (Hex)	Node	DLC	Data Payload (Hex)	Decoded Physical Values
8.000	0x110	BMS	8	...	MESSAGE STOPS TRANSMITTING
8.220	0x120	VCU	8	00 00 00 01 00 00 00 00	Torque Cmd: 0.0 Nm, Inv En: 0, Mode: 1
8.240	0x130	MCU	8	00 00 C0 0F 55 5A 00 FA	Act Torque: 0.00 Nm, RPM: 4032, InvT: 45°C, MotT: 50°C, RC: 250

Analysis:

1. **Watchdog Trigger:** The VCU listener thread maintains a delta-time counter between consecutive 0x110 frames. Once the timer eclipses the hardcoded 200 ms limit, a timeout exception logic branch is executed.
2. **Safe State Execution:** The VCU voids the driver's throttle input entirely, dropping the torque command to 0 Nm and toggling the Inverter Enable byte to 0.
3. The MCU enters an immediate free-wheeling or coasting state, demonstrating a critical fail-safe dependency natively orchestrated over the serial bus. The vehicle safely removes propulsion power, avoiding unpredictable behavior from a blind master controller.

Challenges and Limitations

While the software-only CAN network provides deep analytical insight into the electric powertrain, the implementation faces several intrinsic limitations tied to the runtime environment and abstraction models.

1. **Python Concurrency vs. Real-Time Determinism:** The standard Python runtime utilizes a Global Interpreter Lock (GIL). Although using OS threads via the threading library combined with `can.ThreadSafeBus` allows concurrent execution, Python operates strictly under soft real-time constraints. It is continuously subject to the unpredictable task scheduling latency of the host Linux operating system. In production high-speed automotive systems, ECUs operate on dedicated microcontrollers utilizing preemptive Real-Time Operating Systems (RTOS) like FreeRTOS or Zephyr, ensuring hard millisecond timing. During system stress, the Python simulation occasionally exhibits cycle time jitter (e.g., a 20ms frame transmitting at 24ms).
2. **Virtual Bus Limitations:** While `vcn` is excellent for higher-level application logic validation, it completely abstracts the physical layer (PHY). The simulation cannot natively model electrical bus arbitration failures, cyclic redundancy check (CRC) bit stuffing errors, electromagnetic interference (EMI) corruption, or physical bus capacitance issues that drastically affect real-world CAN integrity.
3. **Sensor and Mathematical Fidelity:** The simulation relies on rudimentary Euler numerical integration for physics modeling. Real MCU Field Oriented Control involves high-frequency (typically 10 kHz to 20 kHz) Park and Clarke mathematical transformations. The simulated MCU abstractions provide sufficient macroscopic operational data (Torque and RPM) but cannot be utilized to tune low-level Phase-A/B/C current loops or inverter switching dead-times.

Conclusion

The complete architectural simulation of an Electric Vehicle CAN bus demonstrates the profound complexity and interconnected nature of modern vehicular embedded networks. By successfully linking the Battery Management System, Vehicle Control Unit, and Motor Control Unit via a Linux `vcn` topology, the project validates critical software integrations required for safe EV operation. The multi-threaded Python implementation effectively bridges the gap between high-level physical powertrain modeling and low-level serial communication protocols, ensuring deterministic byte-packing and robust handling of cyclic frame transmission.

Through the implementation of mathematically derived physical states and the execution of rigorous drive permission logic, the system successfully translates dynamic throttle mapping while strictly adhering to the electro-chemical safety boundaries of the high-voltage battery pack. Furthermore, the successful mitigation of simulated fault events—specifically thermal runaway derating and bus communication timeouts—highlights the resilience of a correctly implemented VCU supervisory framework. Despite the soft real-time constraints inherently imposed by the Python execution environment, this virtual telemetry architecture provides a highly scalable, rigorously defined test bench, establishing a foundational toolchain for advanced algorithm validation, continuous integration testing, and Software-in-the-Loop powertrain development.

Appendix: Version Control History and Execution Instructions

In adherence to the software lifecycle tracking requirements , the development of this simulation architecture was managed via a structured version control repository.

Simulated Git Log Repository History

Repository Name: arys-ev-can-simulation

Commit Hash	Author Date	Commit Message
a1b2c3d	2026-05-20	Initial commit: Added README.md outlining project scope
f4e5d6c	2026-05-21	feat: Setup virtual CAN environment and ThreadSafeBus wrapper
b7a8c9d	2026-05-21	feat: Implement base ECUNode threading class with timeout logic
e1f2a3b	2026-05-22	feat: Add BMS physical models (Coulomb counting & Joule heating)
c4d5e6f	2026-05-22	feat: Implement VCU drive permission and CAN struct packing
9a8b7c6	2026-05-23	fix: Resolve GIL deadlock during shutdown using threading.Event
d5c4b3a	2026-05-23	test: Add scenario scripting for thermal runaway and timeouts
1a2b3c4	2026-05-23	docs: Finalize code comments and physical equation documentation

Environment Setup and Execution Instructions

To execute the provided Python source code and reproduce the CAN telemetry results, the host Linux machine must be properly configured to route virtual socket traffic.

1. Virtual Hardware Initialization: Open a terminal and execute the following commands to install the kernel modules and instantiate the vcan0 interface :

```
sudo modprobe can
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
```

2. Python Environment Configuration: Ensure Python 3.10+ is installed on the host system. Install the required external library utilizing pip:

```
pip install python-can
```

3. Application Execution: Save the embedded source code provided in the Implementation Details section into a file named `main_vcu_sim.py`. Execute the script from the terminal:

```
python3 main_vcu_sim.py
```

4. Real-Time Bus Monitoring (Optional): To observe the raw hexadecimal traffic traversing the virtual bus in real-time alongside the application output, open a secondary terminal window and utilize the `candump` utility provided by the `can-utils` Linux package:

```
candump vcan0
```

This utility will visibly output the 0x110, 0x111, 0x120, and 0x130 frames as they are simultaneously packed and unpacked by the concurrent Python threads.

Works cited

1. New Energy Vehicle VCU, BMS, MCU Controller Diagram - EEWorld.
<https://en.eeworld.com.cn/news/qrs/eic699164.html>
2. An Overview of Electric Vehicle Motor Control Unit | Dorleco. <https://dorleco.com/an-overview-of-electric-vehicle-motor-control-unit/>
3. CAN Protocol Communication System with MRS Developers Studio.
https://www.researchgate.net/publication/376527096_CAN_Protocol_Communication_System_with_MRS_Developers_Studio_in_ATV_Electric_Vehicles
4. python-can 4.6.1 documentation. <https://python-can.readthedocs.io/>
5. 4.6.0 • pypi-python-can • tessel • Registry • Tessel. <https://tessel.io/registry/tessel/pypi-python-can/files/docs/hardware-interfaces.md>
6. Harnessing Python and ChatGPT for Automotive Security Research.
<https://vicone.com/blog/harnessing-python-and-chatgpt-for-automotive-security-research>
7. scapy/doc/scapy/layers/automotive.rst at master - GitHub.
<https://github.com/secdev/scapy/blob/master/doc/scapy/layers/automotive.rst>
8. Can more than one processes be run using threads in Python? If so, how.
<https://www.quora.com/Can-more-than-one-processes-be-run-using-threads-in-Python-If-so-how>
9. Multithreaded Python: Slithering Through an I/O Bottleneck - Medium.
<https://medium.com/better-programming/multithreaded-python-slithering-through-an-i-o-bottleneck-65250022b467>
10. Getting Started with CAN(Controller Area Network) in Linux?
<https://thetechwin.wordpress.com/2019/03/15/getting-started-with-cancontroller-area-network-in-linux/>
11. cancli - PyPI. <https://pypi.org/project/cancli/>
12. Simulation platform - Guidance for Connected Mobility on AWS.
https://docs.aws.amazon.com/zh_cn/guidance/latest/connected-mobility-on-aws/simulation-platform.html
13. Python for EV Engineers: Skills Roadmap from Scripting to Data.
<https://emobility.careers/python-for-ev-engineers-skills-roadmap>
14. What Is Python Used for in Automotive Engineering? - A&D Company.
<https://aanddtech.com/2025/05/28/python-used-automotive-engineering/>
15. PDF - Python-CAN. https://python-can.readthedocs.io/_downloads/en/main/pdf/
16. Automated Tool for CAN Bus Message Mapping - DSpace Repository.
<https://dspace.cvut.cz/bitstreams/4b8d3533-8f08-45ed-94b5-fccfe9d3e25f/download>
17. Add Virtual Threads to Python - Page 11 - Ideas. <https://discuss.python.org/t/add-virtual-threads-to-python/91403?page=11>
18. Help with reading multiple CAN messages with python.

- <https://forums.raspberrypi.com/viewtopic.php?t=372824>
19. Bus — python-can 3.3.3 documentation. <https://python-can.readthedocs.io/en/3.3.3/bus.html>
 20. Bus — python-can 4.0.0 documentation. <https://python-can.readthedocs.io/en/4.0.0/bus.html>
 21. Seamless Multithreading with Python Events: A Complete Guide. <https://rahulsreedharan.medium.com/seamless-multithreading-with-python-events-a-complete-guide-a1c5a3457e26>
 22. Product Explorer - SPREAD.ai. <https://www.spread.ai/product-explorer>
 23. 特斯拉Model 3/Y CAN总线DBC文件：解锁200+车辆信号的完整技术. https://blog.csdn.net/gitblog_01126/article/details/160499387
 24. Access underlying python-can bus · Issue #19 - GitHub. <https://github.com/pylessard/python-can-isotp/issues/19>
 25. threaded python-can suddenly stops sending messages. <https://stackoverflow.com/questions/77623250/threaded-python-can-suddenly-stops-sending-messages>
 26. CAN Bus in BMS: Why Communication Matters - NX Technologies. <https://nx-tech.com/insight-hub/can-bus-bms/>
 27. Role of the VCU in Electric Vehicles - ALLPCB. <https://www.allpcb.com/allelectrohub/role-of-the-vcu-in-electric-vehicles>
 28. VCU vs MCU: How Does it in Electric Vehicles Function Hand in Hand. <https://www.embitel.com/blog/embedded-blog/how-do-vcu-and-mcu-in-electric-vehicles-function-hand-in-hand>
 29. (PDF) Predictive Health Monitoring Systems for Electric Vehicle Powertrains Using Edge AI and CAN Bus Data. https://www.researchgate.net/publication/399873911_Predictive_Health_Monitoring_Systems_for_Electric_Vehicle_Powertrains_Using_Edge_AI_and_CAN_Bus_Data
 30. A Real-Time In-Vehicle Network Testbed for Machine Learning. <https://ceur-ws.org/Vol-3125/paper5.pdf>
 31. 从零开始学CANoe系列：CANdb++创建DBC文件全解析 - 百度智能云. <https://cloud.baidu.com/article/4172766>
 32. python-can: Would like to send several messages at the same. <https://github.com/hardbyte/python-can/discussions/1964>
 33. 特斯拉Model 3/Y CAN总线协议解析：汽车电子数据框架与实战指南. https://blog.csdn.net/gitblog_00846/article/details/160499379
 34. Study on Simulation and Debugging of Electric Vehicle Control System. <https://www.mdpi.com/2032-6653/17/2/57>
 35. Add Virtual Threads to Python - Page 7 - Ideas. <https://discuss.python.org/t/add-virtual-threads-to-python/91403?page=7>
 36. Add Virtual Threads to Python - Page 4 - Ideas. <https://discuss.python.org/t/add-virtual-threads-to-python/91403?page=4>
 37. COMP4801 Final Report. <https://wp2024.cs.hku.hk/fyp24001/wpcontent/uploads/sites/2/2024/09/COMP4801-Final-Report-2.pdf>